

Ransomware Detection Using Printable Strings

Avinash Singh

*Department of Computer Science
University of Pretoria
Pretoria, South Africa
asingh@cs.up.ac.za
0000-0003-3024-4076*

Daneel Nortier

*Department of Computer Science
University of Pretoria
Pretoria, South Africa
u18019634@tuks.co.za*

Richard Adeyemi Ikuesan

*College of Technological innovation
Zayed University
Abu Dhabi, United Arab Emirates
richard.ikuesan@zu.ac.ae
0000-0001-7355-2314*

Abstract—Ransomware is a type of malware that blocks access to some or all the files on the infected machine until a ransom is paid to the attackers. Ransomware is one of the fastest-growing and most costly types of malware, increasing the need for a fast and accurate ransomware detection framework. Executable files are a common ransomware attack vector, and many users interact with executable files regularly. Traditional ransomware detection methods remain ineffective against novel ransomware attacks, and dynamic analysis techniques require an impractical amount of time and resources for ransomware detection. This work attempts to address these issues by evaluating the efficacy of applying machine learning and natural language processing techniques to the strings extracted from executable files, which is a static analysis technique. Term frequency-inverse document frequency, Bag-of-words and Doc2Vec natural language processing techniques were explored to preprocess the data before utilizing the machine learning techniques. The support vector machine, random forest, adaboost, k-nearest neighbors and decision tree classifiers were used to classify the executable as either benign or ransomware. The proposed model provided a fast and accurate method for the classification of ransomware in executable files with an accuracy of 99.38% and a prediction time of 0.0366ms per file. This research aids the community in using a faster approach to ransomware detection before more resource-intensive detection mechanisms are used. In addition to the supervised classifiers above, the work extends the framework with a small reinforcement learning (RL) experiment that: (1) selects the best-performing supervised model on a validation split, (2) uses that model to bootstrap a lightweight policy via behavioural cloning, and (3) further optimises the policy with a simple REINFORCE algorithm using the dataset. The RL policy achieves competitive test performance (accuracy 0.93, macro F1 0.93) while providing a path for online or continual adaptation in deployment scenarios.

Index Terms—Ransomware Detection, Machine Learning, Natural Language Processing, TF-IDF, Static Analysis, Strings.

I. INTRODUCTION

Ransomware is a type of malware that denies access to some or all of the files on the infected machine. After a machine is infected by ransomware, the attackers then demand payment from the user whose machine is infected in order to regain access to their files [1]. Ransomware attacks come in two forms, Locker Ransomware and Crypto Ransomware [2]. In the case of Locker Ransomware, the user is prevented from logging into their device, thus rendering the files inaccessible, but the files are not encrypted. In the case of Crypto ransomware, the target files are encrypted,

thereby rendering them inaccessible to the legitimate user.

Ransomware remains one of the fastest-growing cybercrimes, having increased 259% [3]. In addition, ransomware attacks are one of the most costly types of malware attacks, with the average cost of an attack being \$5.13M in 2024 [4]. Executable files in particular remain one of the most prominently used attack vectors, making up more than a quarter of the malicious email attachments. Ransomware attackers also often employ a double extortion strategy in which attackers copy the infected users' data before the ransomware is deployed. They then demand payment for the victim to regain access to their files, and another payment to delete the copies that the attackers created. If the victim does not pay the second ransom, the attackers threaten to leak the data which they copied [5].

There are two primary techniques for ransomware detection when using ML techniques: dynamic and static analysis. Dynamic analysis is where the suspected Ransomware is executed in a safe environment and its behaviour is analyzed [6] [7] [8]. Relevant information from executable files can be extracted quickly, and using ML and Natural Language Processing (NLP) techniques can provide a fast classification of the executable as ransomware or benign. Static analysis techniques can still maintain high accuracy when detecting ransomware making it ideal for fast ransomware classification in a real-world environment.

II. RELATED WORK

ML techniques are often used in proposed malware detection frameworks due to their capacity to capture and use the underlying patterns in the provided data, making them effective tools for detecting not only known malware but also novel malware [9] [10] [11]. NLP techniques are also frequently employed in proposed malware detection frameworks due to their efficacy in extracting meaning and contextual information from data extracted from executable files [12] [13]. Both ML and NLP techniques are useful for dynamic and static malware analysis frameworks. Dynamic analysis frameworks are generally more effective, overcoming problems such as code obfuscation, but take more time and introduce additional overhead. Static analysis frameworks are generally much

faster, but can sometimes suffer in terms of accuracy due to obfuscation techniques. This section evaluates the literature that has been published on malware and ransomware detection that uses ML and NLP techniques.

A. Dynamic Analysis Methods

There are several systems and tools already created that make use of various machine learning and artificial intelligence techniques to identify ransomware. Lowe et al. [14] leverages semantic analysis of memory opcode patterns, utilizing advanced machine learning models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs). This method significantly improves the accuracy and reliability of ransomware detection by capturing complex semantic relationships within opcode sequences. The study highlights the limitations of traditional detection methodologies, such as signature-based and heuristic approaches, and demonstrates the potential of semantic analysis to address these challenges. The research also identifies key obstacles, including the computational demands of deep learning models and the difficulty in distinguishing between closely related ransomware and benign software. The authors also explored traditional models like RF and SVM. The results varied from 85.6% to 92.8% using a RF and CNN, respectively. While this is a good result ransomware authors tend to mask and use heavy obfuscation techniques, which makes this model easy for attackers to bypass the detected patterns.

Ahmed and Al-Dabbagh (2021) [11] explored the detection of ransomware using ML techniques on mobile devices running the Android operating system. The framework utilises the network traffic, which is monitored over time, of the applications on the device as the data for the ML classifiers. The network flow features that were used consisted of six columns for each flow: FlowID, SourceIP, DestinationIP, SourcePort, DestinationPort, and Protocol. In addition, 79 network traffic features were also used. 603288 samples of the network traffic of ransomware and benign applications were included in the dataset. Six (6) ML classifiers were trained and tested on the data to compare their efficacy. The classifiers used are Decision Tree, Random Forest, Logistic Regression, K-Nearest Neighbour, XGBoost and Multi-Layer Perceptron. The system managed to achieve a precision of over 90% in all ML classifiers except for the Multi-Layer Perceptron and Logistic Regression classifiers, and the Decision Tree and XGBoost classifiers also achieved an accuracy of over 99%. The proposed framework achieves high overall accuracy and targets ransomware on Android systems, which are the most popular smartphone operating system, however, it does fall short in several areas. The framework requires the applications to be executed in either a live environment, opening the system up to attack, or a sandbox environment that has to be set up. All application traffic on the device then needs to be monitored for some time before sufficient network traffic information can be extracted. This makes the proposed framework slow

to detect ransomware and impractical for use in a real-world scenario.

B. Static Analysis Methods

The framework achieves high accuracy and can provide a relatively fast classification of ransomware due to its static analysis nature, however it does have a few flaws. Due to the extreme length of opcode sequences, additional overhead is introduced in that a multitude of self-attention convolution neural networks need to be introduced to be applied to each patch in the opcode sequence, and each of these networks needs to be trained every time new data is added into the system.

Mimura and Ito (2022) [13] utilises NLP and ML techniques applied to the strings extracted from executable files for malware detection. The dataset consists of over 250,000 benign executable files and over 250,000 malicious executable files, collected from various sources. Two NLP techniques are applied to these printable strings as preprocessing for the ML techniques, namely Doc2Vec and Latent Semantic Indexing. Latent Semantic Indexing is an NLP technique that analyzes the relevance between a corpus and the words in a specific document. The feature vectors created by the NLP techniques are fed into one of 5 ML classifiers, namely Support Vector Machine, Random Forest, XGBoost, Multilayer Perceptron or a Convolutional Neural Network. The framework achieved an f1 score of over 90% irrespective of the ML or NLP technique used, or any combination of them. Due to the static nature of the framework, the ML classifiers can be trained and run in very short periods. Despite this, the framework is lacking in several areas. Although a large number of malicious and benign files were collected, when training and testing the classifiers, the number of files used was very skewed towards benign files. In addition, the scope of the framework is extremely broad, including several different malware families without any comparison of the framework's performance when applied to different malware categories. This broad scope also means that the framework is unlikely to be effective at detecting novel strains of malware in any one specific family.

Khammas (2020) [15] proposed a novel static analysis framework to detect ransomware in executable files by using frequent pattern mining and the Random Forest ML classifier. The dataset consists of 1680 executable files, with half of them being ransomware and the other half being benign files. The data pre-processing was done in three stages to extract as much useful information as possible. First, the raw bytes were extracted in a virtual machine using a 32-bit sliding window. Afterwards, the Gain Ratio feature selection method is applied, and finally, the frequent patterns are normalised. A comparison of the accuracy of the Random Forest when using differing amounts of features, from 1000 up to 7000, was done, with the best performance being achieved while using 1000 features. A comparison of the number of trees

used was also done, with 100 trees finally being determined to be the best at classification within a reasonable amount of time. The classifier was able to achieve an accuracy of 97.74% after tuning. The proposed framework was able to achieve high accuracy, however, the proposed framework is lacking in several areas. The framework makes no comparison of the performance of different preprocessing techniques, and it makes no comparison of different ML classifiers. Due to this fact, the merit of the underlying technique is not properly explored, and there could be other techniques that could provide better overall performance. In addition, the three-tiered approach to feature extraction introduces additional overhead to the system that reduces the overall performance.

III. DETECTING RANSOMWARE WITH STRINGS

This work evaluates the potential efficacy of using various ML and NLP techniques applied to the strings extracted from executable files to detect whether an executable file contains Ransomware. This work focuses only on the binary classification of an executable as either a benign file or a file containing Ransomware and does not consider a multi-class classification of more than one type of malware. The evaluation of the static analysis method of the ransomware files has been done on the strings extracted from the ransomware and benign files. These strings were extracted from raw Cuckoo Sandbox reports using the MalFe platform [16]. The extracted strings are then processed using NLP techniques to attempt to extract relational, contextual and individual meaning from the strings in a file. In addition, these NLP techniques serve as feature generators for the ML techniques that were explored in this research. Three differing NLP techniques have been implemented to compare their efficacy in the problem domain. TF-IDF is an NLP technique that compares the frequency of a word or phrase appearing in a document to the inverse of the frequency of the word or phrase appearing in the entire corpus. Second, Bag-of-Words has been implemented, which is an NLP technique that focuses on only the frequency of words or phrases that appear in a specific text body or "bag". Finally, Doc2Vec has been implemented, which is an NLP technique that extends the Word2Vec technique by generating a feature vector for all the documents in a corpus. The feature vector outputs from these NLP techniques are then fed into five (5) ML classifiers. These classifiers are used to classify the input files as either benign or ransomware, following a binary classification problem. These ML classifiers are then trained on the feature vector outputs from the NLP techniques, and their parameters are tuned to try and improve their accuracy.

The proposed method for detecting ransomware in executable files is broadly outlined in Figure 1. The process starts with the extraction of the strings from executable files using already analysed Cuckoo Sandbox reports. The extracted strings are all of the printable strings that are found within the executable file. The collected strings are processed using one of three NLP techniques: TF-IDF, Bag-of-Words or Doc2Vec.

The resulting feature vectors are the input for the ML classifiers that finally predict whether or not an executable contains ransomware. The proceeding subsections discuss each of the steps outlined in the process.

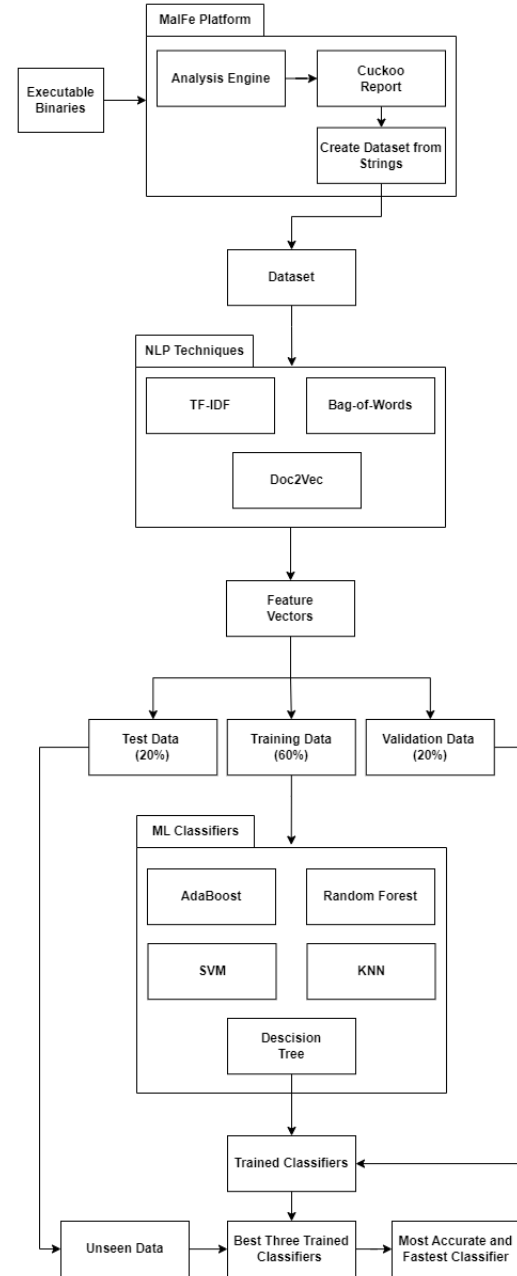


Fig. 1. Process Model

A. NLP Techniques

NLP techniques are a way to convert a corpus of words into a real-valued vector in such a way that the semantics and relationships between the words are captured [17].

In this research, the strings were extracted from Cuckoo sandbox reports from the MalFe platform [16]. The strings of a file are extracted by finding matches with the two regular

expressions, namely **b”[\x1f-\x7e]6,”** and **b”(?:[\x1f-\x7e][\x00])6,”** [18]. The strings are then capped at a length of 1024 characters, and a maximum of 2048 strings are extracted from a file. Some examples of matched strings are: **”VWUUhP@@,”**, **”The unexpected small block leaks are:,”**, **”EINVALOp”** and **”Inno Setup Messages (5.5.0) (u)”**.

1) *Term Frequency-Inverse Document Frequency*: TF-IDF is an NLP technique that considers the frequency of the appearance of a word in a document and divides this by the number of documents in the corpus that the word appears in. It can be formulated as follows:

$$TF = \frac{\text{number of times the term appears in the document}}{\text{total number of terms in the document}}$$

$$IDF = \log\left(\frac{\text{number of the documents in the corpus}}{\text{number of documents in the corpus that contain the term}}\right)$$

$$TF - IDF = TF * IDF$$

Thus, it measures how important a word is within a specific document [19]. So as an example, consider a corpus of three documents:

Document 1: *’data science is one of the most important fields of science’*

Document 2: *’this is one of the best data science courses’*

Document 3: *’data scientists analyze data’*

In the first document, the first word is ‘data’, since the word appears once in the document and there are 11 words in total in the document, the $TF = \frac{1}{11} = 0.090909$. Then, since there are 3 documents in the corpus and the term ‘data’ appears in all three, $IDF = \log\left(\frac{3}{3}\right) = 0$, thus the TF-IDF term for the term ‘data’ in the first document would be $0.090909 * 0 = 0$. Calculating this for each of the terms in the corpus with respect to the first document would yield the vector representation: [0.043375, 0.000000, 0.000000, 0.000000, 0.000000, 0.000000, 0.032017, 0.043375, 0.016008, 0.016008, 0.032017, 0.043375, 0.016008, 0.0]. This process can then be completed for each of the other documents in the corpus to get a vector of three vectors, each of the same length as the total number of unique words in the corpus.

2) *Bag-of-words*: Bag-of-words is an NLP technique representing each document in the corpus as a vector of a fixed length, equal to the total vocabulary size. If the word corresponding to the index in the vector is present in the document, it is represented by a one (1); otherwise, it is represented by a zero (0) [20]. As an example, consider a corpus of four documents as follows:

Document 1: *’It was the best of times’*

Document 2: *’it was the worst of times’*

Document 3: *’it was the age of wisdom’*

Document 4: *’it was the age of foolishness’*

Next, take all of the unique words in any order, for example: “it”, “was”, “the”, “best”, “of”, “times”, “worst”, “age”, “wisdom” and “foolishness”. The first document would then have a vector representation of: [1, 1, 1, 1, 1, 1, 0, 0, 0, 0] since the first five words in the list of unique words in the

corpus appear in the document, while the last four do not [21].

3) *Doc2Vec*: Doc2Vec is an NLP technique that extends the popular Word2vec NLP technique by adding a paragraph vector. Thus, instead of creating a vector of a word, it encodes an entire document into a vector [22]. Word2Vec works by training a simple neural network with a single hidden layer on the corpus of data; the embedding that is then created in the hidden layer can then be used as a representation of the corpus. Doc2Vec extends this by adding another paragraph ID vector that is unique to each document. These two vectors are then concatenated which results in the final vector representation of each of the documents in the corpus [23]. Since the number of neurons in the hidden layers of the neural networks can be controlled, the size of the vector representation can be controlled. As an example, if we take the corpus:

Document 1: *’Come end of fall, the winds will grow’*

Document 2: *’then twisting branches will cusp the snow’*

Document 3: *’but spring will free what cold held dear’*

Document 4: *’give way, give way, for one more year’*

The Doc2Vec representation of the first document with a vector size of 10 would be [-0.04018994, 0.0330957, 0.02324159, -0.0238918, -0.0057377, 0.02368324, -0.02787386, -0.02250209, 0.04231475, 0.01566228]

B. Dataset

The data consisted of 3325 executable files of which 1497 files are benign, and 1828 are malicious. These executable samples were sourced from TheZoo, PortableApps, Dike-Dataset and MalwareBazaar. These strings were extracted from raw Cuckoo Sandbox reports using the MalFe platform [16]. The raw dataset can be found with the name “Ransomware Detection Using Strings”¹ on the MalFe platform. The distribution of the categories of data is demonstrated in Table I. In total, the files contained millions of strings, with an average of 1501 strings per file. The executable files strings contain valuable information about the executable and, as such, they are treated as natural language strings. This allows for positional and relational information about the strings to be encoded and increases the efficacy of the ML techniques. The next step is to pre-process this data by feeding it into NLP techniques.

¹<https://malfe.cs.up.ac.za/datasets/view/RDS/60#version1>

TABLE I
DATASET CATEGORIES.

Category	Count
Ransomware	1828
Utilities	223
Internet	86
Graphics	55
Office	54
Games	52
Music / Video	46
Security	30
Development	20
Education	9
Benign (Other)	970

C. Data Quality and Statistics

Printable strings were extracted statically from each binary and aggregated per sample. The average string length per sample differs substantially between classes:

- Benign samples: mean average string length of 2453 characters
- Ransomware samples: mean average string length of 4224 characters

This difference suggests that ransomware binaries tend to embed longer printable string sequences, which may include encrypted payload fragments, embedded resources, or statically linked metadata. Importantly, this observation reflects string length, not string count, indicating that ransomware often contains fewer but substantially longer printable regions. The number of long strings (defined as strings exceeding 20 characters) also varies between classes. Benign samples exhibit a higher mean count of long strings (42 per sample) compared to ransomware (27 per sample). This indicates that benign software tends to include numerous medium-length, human-readable strings (e.g., API names, file paths, UI text), whereas ransomware strings are more skewed toward fewer, very long sequences.

To assess the semantic quality of extracted strings, Shannon entropy was computed per string and averaged per sample. The results show comparable entropy levels across classes:

- Benign: mean entropy 4.71
- Ransomware: mean entropy 4.56

Both classes fall within a mid-entropy regime, indicating that the dataset is not dominated by either low-entropy boilerplate text or high-entropy encrypted noise. The slightly lower entropy observed in ransomware strings suggests the presence of structured, repetitive content (e.g., manifest metadata, API names, privilege requests) alongside obfuscated regions. These results confirm that the extracted printable strings retain meaningful semantic structure, supporting their suitability for downstream statistical and machine-learning analysis.

To evaluate robustness to noise and low-information strings, strings shorter than increasing minimum lengths were progressively removed. The average number of retained strings per

sample decreases smoothly with stricter thresholds for both classes:

- With a minimum length of 4 characters, benign samples retain 49 strings on average, compared to 37 for ransomware.
- At a minimum length of 16 characters, benign samples retain 44 strings, while ransomware retain 28.

The monotonic and gradual decline indicates that the dataset does not rely on a small number of short, fragile indicators. Instead, discriminative information persists even after aggressive pruning, suggesting that the printable strings exhibit robust signal density rather than brittle keyword dependence.

To quantify the informativeness of individual strings, mutual information (MI) was computed between binary string presence and the ransomware label. Highly ranked strings include:

- Structural PE sections (e.g., .data, .rdata)
- Windows API references (e.g., kernel32.dll, GetLastError)
- Application manifests and privilege descriptors (e.g., requestedExecutionLevel, trustInfo)
- Configuration and metadata terms (e.g., assembly, manifestVersion, security)

The diversity of high-MI strings indicates that discrimination is not driven by a single keyword or artefact, but rather by a broad spectrum of semantically meaningful indicators spanning binary structure, execution privileges, and system interaction. The top 20 strings are presented in Table II

TABLE II
MI OF TOP 20 STRINGS.

String	MI
version	0.599293
.data	0.562157
kernel32.dll	0.542670
name	0.542467
.rdata	0.501461
assembly	0.488292
type	0.483308
security	0.481017
level	0.476413
GetCurrentProcess	0.471400
xmlns	0.470347
false	0.466049
schemas-microsoft-com	0.464501
manifestVersion	0.464119
asm.v1	0.463737
GetLastError	0.461255
trustInfo	0.458204
requestedPrivileges	0.457824
requestedExecutionLevel	0.457824
system	0.454032

Overall, the dataset exhibits several desirable quality properties:

- Balanced and heterogeneous class composition, reducing sampling bias.

- Non-trivial string length and entropy distributions, indicating meaningful printable content.
- Robustness to string pruning, suggesting resilience to noise and obfuscation.
- A long tail of informative strings, rather than reliance on a small number of signatures.

These characteristics support the use of printable strings as a reliable and interpretable feature source for ransomware detection and justify their evaluation in subsequent classification experiments.

D. Data Pre-processing

The extracted strings were converted into feature vectors using three NLP techniques: TF-IDF, Bag-of-Words and Doc2Vec. TF-IDF converted the corpus of strings of all the executables into a 2D matrix with 1479647 features representing each input document. Bag-of-Words converts the corpus into a 2D matrix with 664140 features representing each document. Doc2Vec is trained on the input data and can produce a vector of any specified length to represent an executable's strings. An array length of 1000 was used. Now that the NLP techniques were applied separately to the dataset essentially providing a dataset for each technique. Each dataset was then split into 60% training, 20% validation, and 20% testing. The details of the dataset split and features are shown in Table III. This dataset is heavily imbalanced, however, this was due to the limited benign samples that the platform had [16].

TABLE III
NLP DATA METRICS.

	Bag-of-Words	TF-IDF	Doc2Vec
Features	804801	1711443	1000
Fit time (min)	0.1260	0.1670	7.9008
Training Length	2026	2026	2026
Training Malicious	1098	1098	1098
Training Benign	928	928	928
Validation Length	675	675	675
Validation Malicious	366	366	366
Validation Benign	309	309	309
Test Length	676	676	676
Test Malicious	366	366	366
Test Benign	310	310	310

E. Machine Learning Classifiers

The feature vectors from the pre-processors are then fed into ML classifiers for training. The five (5) ML classifiers selected are Decision Tree (DT), Random Forest (RF), AdaBoost (AB), Support Vector Machine (SVM), and K-Nearest-Neighbours (KNN). These classifiers were selected due to the nature of the dataset as well as for their performance in terms of training and prediction times. The goal is to have fast training and prediction times such that the classifier can be deployed in a real-world environment. Therefore, RF and DT were chosen to make sense of the data through rule-based deductions. Boosted learning, such as AB, was also explored to see if more deductions can be

made by evolving the classifier over a number of iterations. SVM and KNN were also considered to see if hyperplanes or clustering could perform better. All the training of the ML classifiers underwent hyperparameter tuning as well as five-fold cross-validation. This was done to ensure the best model parameters were chosen, as well as to ensure there is no bias and noise in the model by performing cross-validation. A variety of metrics were used to evaluate the performance of the system, such as True Positive(TP), False Positive (FP), True Negative (TN), False Negative (FN), Recall, Precision, F1-score, Accuracy, Area Under the Curve (AUC), logloss and latency. The AUC and logloss, on the other hand, define the models correctness and confidence in its prediction. Finally, the latency is the time taken to perform the prediction for the specified model. The source code, results and processed dataset are available at the RDS GitHub repository [24].

1) *Decision Tree*: The Decision Tree (DT) classifier is a model that classifies data points using a series of if-else rules organized in a tree structure. It is easy to understand and interpret due to its visual nature. Additionally, DTs typically perform well with minimal data pre-processing. The best-performing configuration was achieved using the Bag-of-Words representation, with the parameters `criterion='log_loss'` and `splitter='best'`. Among all preprocessing methods, TF-IDF produced the highest accuracy of 94.96% and the lowest false positives and false negatives, indicating it was the most effective configuration overall (see Table V). The default values were used for all other parameters.

TABLE IV
METRICS FOR THE DECISION TREE CLASSIFIER.

	TP	TN	FP	FN	Recall	Precision	F1-Score
Bag-of-Words	293	348	18	16	0.9508	0.956	0.9534
TF-IDF	293	348	18	16	0.9508	0.956	0.9534
Doc2Vec	208	266	100	101	0.7268	0.7248	0.7258

TABLE V
PERFORMANCE METRICS FOR THE DECISION TREE CLASSIFIER.

	Accuracy	AUC	Log Loss	Latency(ms)	Average Latency Per Sample (ms)
Bag-of-Words	0.9496	0.9495	1.8155	27.5192	0.0408
TF-IDF	0.9496	0.9495	1.8155	33.2804	0.0493
Doc2Vec	0.7022	0.7	10.733	1.1332	0.0017

2) *Random Forest*: The Random Forest (RF) classifier is a meta-estimator that builds multiple decision trees on random subsets of the training data and aggregates their predictions to improve accuracy and reduce overfitting. The best-performing configuration was achieved using the Bag-of-Words representation, with `max_depth = 8` and `n_estimators = 5`, yielding a mean test score of 0.9917 and a standard deviation of 0.0104. As shown in Table VII, the Bag-of-Words approach achieved the highest accuracy of 99.16%, with zero false

positives and a fast prediction time of 0.1932ms. All other hyperparameters were set to their default values.

TABLE VI
METRICS FOR THE RANDOM FOREST CLASSIFIER.

	TP	TN	FP	FN	Recall	Precision	F1-Score
Bag-of-Words	295	356	10	14	0.9727	0.9622	0.9674
TF-IDF	295	357	9	14	0.9754	0.9623	0.9688
Doc2Vec	252	311	55	57	0.8497	0.8451	0.8474

TABLE VII
PERFORMANCE METRICS FOR THE RANDOM FOREST CLASSIFIER.

	Accuracy	AUC	Log Loss	Latency(ms)	Average Latency Per Sample (ms)
Bag-of-Words	0.9644	0.9949	0.1211	288.6295	0.4276
TF-IDF	0.9659	0.9959	0.105	2331.629	3.4543
Doc2Vec	0.8341	0.913	0.4363	22.3072	0.033

3) *AdaBoost*: The AdaBoost (AB) classifier is a meta-estimator that initially trains a base classifier and iteratively fits additional classifiers to better handle difficult cases. The best performance was achieved using the TF-IDF representation with parameters `learning_rate = 0.1` and `n_estimators = 250`. This configuration attained a mean test score of 0.9937 and a standard deviation of 0.0081. As shown in Table IX, TF-IDF achieved over 99% in both recall and accuracy, but exhibited high latency, requiring 8.9791ms to classify a single sample. All other hyperparameters were kept at their default values.

TABLE VIII
METRICS FOR THE ADABOOST CLASSIFIER.

	TP	TN	FP	FN	Recall	Precision	F1-Score
Bag-of-Words	295	356	10	14	0.9727	0.9622	0.9674
TF-IDF	297	359	7	12	0.9809	0.9677	0.9742
Doc2Vec	232	305	61	77	0.8333	0.7984	0.8155

TABLE IX
PERFORMANCE METRICS FOR THE ADABOOST CLASSIFIER.

	Accuracy	AUC	Log Loss	Latency(ms)	Average Latency Per Sample (ms)
Bag-of-Words	0.9644	0.9942	0.5731	7054.8534	10.4516
TF-IDF	0.9719	0.9927	0.5748	10956.3744	16.2317
Doc2Vec	0.7956	0.8788	0.6408	117.5032	0.1741

4) *SVM*: Support Vector Machines (SVM) are supervised learning models that perform well in high-dimensional spaces and remain effective even when the number of samples is smaller than the number of features. The best results were obtained using the TF-IDF representation with parameters `C = 10` and `kernel = 'sigmoid'`, achieving a mean test score of 0.9917 and a standard deviation of 0.0083. As shown in Table XI, this configuration yielded an accuracy above 99% and maintained high recall and low false positives. All other hyperparameters were set to their default values.

TABLE X
METRICS FOR THE SVM CLASSIFIER.

	TP	TN	FP	FN	Recall	Precision	F1-Score
Bag-of-Words	296	354	12	13	0.9672	0.9646	0.9659
TF-IDF	288	360	6	21	0.9836	0.9449	0.9639
Doc2Vec	195	298	68	114	0.8142	0.7233	0.7661

TABLE XI
PERFORMANCE METRICS FOR THE SVM CLASSIFIER.

	Accuracy	AUC	Log Loss	Latency(ms)	Average Latency Per Sample (ms)
Bag-of-Words	0.963	0.9855	0.1376	2480.1817	3.6743
TF-IDF	0.96	0.9884	0.1502	6381.0563	9.4534
Doc2Vec	0.7304	0.802	0.5541	331.9006	0.4917

5) *K-Nearest Neighbors*: K-Nearest Neighbours (KNN) is a non-generalizing learning algorithm that classifies samples based on a majority vote of the k nearest neighbours, where k is a user-defined parameter. The best performance was achieved with the TF-IDF representation using parameters `leaf_size = 5`, `n_neighbors = 5`, `p = 2`, and `weights = 'distance'`, yielding a mean test score of 0.9868 and a standard deviation of 0.0142. As shown in Table XIII, despite achieving relatively high accuracy with TF-IDF, KNN exhibited the poorest overall performance among the classifiers, particularly with the Doc2Vec preprocessor, which resulted in an accuracy of just 82.25%. Additionally, KNN had consistently high false positive rates and the highest prediction latency, taking over 355ms per sample. All other hyperparameters were left at their default values.

TABLE XII
METRICS FOR THE KNN CLASSIFIER.

	TP	TN	FP	FN	Recall	Precision	F1-Score
Bag-of-Words	266	355	11	43	0.9699	0.892	0.9293
TF-IDF	294	340	26	15	0.929	0.9577	0.9431
Doc2Vec	206	309	57	103	0.8443	0.75	0.7943

TABLE XIII
PERFORMANCE METRICS FOR THE KNN CLASSIFIER.

	Accuracy	AUC	Log Loss	Latency(ms)	Average Latency Per Sample (ms)
Bag-of-Words	0.92	0.9732	0.7809	796.4272	1.1799
TF-IDF	0.9393	0.9741	0.7791	438.1878	0.6492
Doc2Vec	0.763	0.8519	1.8379	1776.0074	2.6311

IV. REINFORCEMENT LEARNING EXTENSION

We extended the supervised strings pipeline with a compact reinforcement-learning (RL) experiment to investigate whether a small policy can be bootstrapped from a high-performing supervised detector and subsequently adapted by interaction. The primary aims were: (1) provide a reproducible recipe to convert a static detector into a lightweight policy suitable for online refinement; and (2) evaluate the practical gains and trade-offs of such an approach on the RDS dataset.

A. Environment and reward design

To keep the initial experiment simple and interpretable we framed the dataset as a deterministic, episodic environment. Each episode iterates once through the training split in a fixed order; at each step the agent receives the TF-IDF feature vector for a single file and must return an action $a \in \{0, 1\}$ that corresponds to the predicted label. We use an immediate reward of +1 for a correct label and -1 for an incorrect label. The deterministic traversal reduces environmental variance due to stochastic sampling and isolates policy improvement dynamics.

The reward is intentionally symmetric and sparse (binary correctness) which keeps credit assignment straightforward and aligns the RL objective with supervised accuracy while still allowing policy-gradient updates to shift behaviour based on cumulative episode returns.

B. Policy architecture and pretraining

The policy is a small multilayer perceptron: an input linear layer (dimension = TF-IDF feature length) to a hidden layer of 128 ReLU units, followed by a linear output layer producing two logits. A softmax converts logits into a categorical distribution over actions. The small capacity is chosen to keep inference fast and to reduce overfitting risk on the relatively small training split.

Before running RL updates we perform behavioural cloning: we select the best supervised model on validation (see Table ??) and use it to label the training split. The policy is trained for a small number of epochs (5) with CrossEntropy loss to imitate the teacher. This step provides a strong, high-precision initialization, substantially reducing early random exploration and stabilising policy-gradient updates.

C. Policy-gradient fine-tuning (REINFORCE)

We apply an episodic REINFORCE update loop. Each episode collects log-probabilities of sampled actions and the scalar rewards for each time-step. Returns are computed with discount factor γ ; to reduce variance we normalise the per-episode returns to zero mean and unit variance prior to computing policy gradients. The per-step policy loss is the negative log-probability multiplied by the corresponding normalised return; gradients are accumulated over the episode and applied with the Adam optimiser ($\text{lr} = 1 \times 10^{-3}$). In our experiments we used $\gamma = 0.99$ and 30 episodes.

Normalising returns acts as a simple baseline and reduces the impact of scale differences between episodes; combined with the behaviour-cloning initialization this reduces gradient variance and helps avoid mode collapse or catastrophic policy shifts. The deterministic environment also prevents variance caused by random sampling order.

D. Mitigating bias and overfitting

We took several pragmatic steps to reduce bias introduced by the RL procedure and to make evaluation fair and comparable to the supervised baseline:

- Model selection for behavioural cloning uses validation F1 (not training loss), ensuring the teacher model generalises to held-out data before bootstrapping the policy.
- Behavioural cloning is run for a small, fixed number of epochs (5) to imitate teacher behaviour without fully memorising noisy labels.
- Per-episode normalisation of returns reduces episodic scale differences and prevents large-magnitude gradients dominated by a few episodes.
- The final policy is evaluated only on the held-out test split (never used during selection or RL updates) to provide an unbiased performance estimate.

These measures do not eliminate all sources of bias (for instance, the teacher’s mistakes can be transferred to the policy), but they significantly constrain overfitting and ensure the RL steps act as a refinement rather than an unconstrained re-training.

E. Validation-stage model comparison

Before bootstrapping, we evaluated all candidate supervised models on the validation split and selected the highest-F1 model (AdaBoost on TF-IDF). Table ?? summarises the validation F1 scores for the models considered (extracted from the experiment logs).

TABLE XIV
VALIDATION F1 SCORES FOR CANDIDATE SUPERVISED MODELS
(SELECTION STAGE).

Model	Validation F1
AB_TFIDF	0.97422
AB_BOW	0.96739
RF_TFIDF	0.96879
RF_BOW	0.96739
SVM_TFIDF	0.96386
SVM_BOW	0.96589
DT_TFIDF	0.95342
DT_BOW	0.95342
KNN_TFIDF	0.94313
KNN_BOW	0.92932
RF_DOC2VEC	0.84741
AB_DOC2VEC	0.81551
KNN_DOC2VEC	0.79434
SVM_DOC2VEC	0.76607
DT_DOC2VEC	0.72578

F. RL experimental results

After behavioural cloning and REINFORCE fine-tuning (30 episodes) we evaluated the learned policy on the held-out test

split. Table ?? reproduces the classification report extracted from the test run.

TABLE XV
RL POLICY TEST METRICS (CLASSIFICATION REPORT).

Class	Precision	Recall	F1-score	Support
0 (Benign)	0.90	0.95	0.93	310
1 (Ransomware)	0.95	0.92	0.93	366
Accuracy	0.93			
Macro avg	0.93	0.93	0.93	676
Weighted avg	0.93	0.93	0.93	676

Figure ?? is a placeholder for the RL training curve (episode return or average reward per episode). In our implementation the REINFORCE loop records per-episode cumulative reward and the behavioural-cloning loss; plotting these signals helps visualise convergence and stability. We include placeholders so the figures can be generated after re-running the training loop with logging enabled.

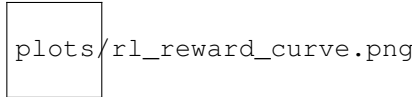


Fig. 2. RL training: cumulative reward per episode (placeholder).

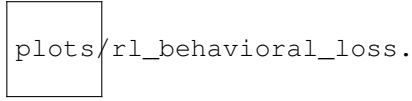


Fig. 3. Behavioural cloning training loss (placeholder).

The RL policy achieves a final test accuracy of 0.93 and macro F1 of 0.93. Compared to the best supervised TF-IDF detectors (validation F1 $\approx$ 0.974), the RL policy is slightly lower in static performance but provides an adaptable policy that can continue to learn from future interactions or delayed labels.

G. Interpretation

Two practical takeaways emerge from the experiment. First, behavioural cloning from a high-quality supervised teacher produces an initial policy that already captures most discriminative structure in the TF-IDF representation; the RL phase acts as a light refinement rather than wholesale re-training. Second, the simple REINFORCE updates (with returns normalisation) are sufficient to preserve high performance while keeping the architecture and training cost modest, making this approach feasible for constrained deployment scenarios where occasional online updates are desired.

V. DISCUSSION

VI. DISCUSSION

Overall, using strings with NLP and ML techniques as a ransomware detection mechanism provides good detection capabilities. In addition, the proposed method provides quick

detection of ransomware; however, the training time was significantly more due to hyper-parameter tuning as well as five-fold cross-validation. On average, using a cluster of 56 logical processors and 378GB RAM, training times were between 0.3 min - 20 min. Prediction times were performed on a standard machine with 8 logical processors delivering times of 25ms - 4000ms. These low training and classification times mean that the system is fast enough that it could be the underpinnings of a system that is deployed in the real world. Typically, prediction times are the priority whilst maintaining high accuracy in order for real-time detection.

The best-performing classifiers are AB with TF-IDF, SVM with BOW and DT with TF-IDF, with accuracies of 99.58%, 99.37% and 99.16% respectively. However, AB with TF-IDF provides prediction times of 8.9791ms, making it worse than SVM and DT in this case. Therefore, the best classifier will be determined by the fastest and most accurate. These three classifiers were then further evaluated to determine the best-performing classifier by performing predictions on testing data to further determine their performance on unseen data. The results for the testing data predictions are demonstrated in table XIV.

TABLE XVI
TESTING DATA PREDICTIONS OF THE THREE BEST PERFORMING CLASSIFIERS.

	AB with TF-IDF	SVM with BOW	DT with TF-IDF
TP	113	113	113
TN	365	364	364
FP	1	2	2
FN	1	1	1
Recall	0.9973	0.9945	0.9945
Precision	0.9973	0.9973	0.9973
F1-Score	0.9973	0.9959	0.9959
AUC	0.9996	0.9993	0.9929
Log Loss	0.3488	0.0327	0.2253
Latency (ms)	167.4294	1671.4294	17.5536
Average latency per sample (ms)	8.6390	3.4821	0.0366
Accuracy	0.9958	0.9938	0.9938

Therefore, from the results, AB with TF-IDF still had the highest accuracy but the slowest prediction times. SVM with BOW and DT with TF-IDF scored an accuracy of 99.38%. Although, the AUC for SVM was slightly better, overall, the faster prediction time was the deciding factor. Therefore, the best classifier is DT with TF-IDF, with an average latency per sample of 0.0366ms. Low FN predictions are ideal since an FN prediction would potentially lead to a user installing ransomware on their system while thinking it to be benign. This is coupled with the fact that the DT classifier trained in 0.1442min, further cements it as the best classifier. Since the dataset was heavily imbalanced, a test was conducted to balance the ransomware samples at random. The results of this can be found in Appendix A. As expected, the results differed slightly by approximately 2% in accuracy. However, the other metrics are close to the results obtained from the unbalanced dataset.

To visualize the diagnostic ability of the classifiers, the Receiver Operating Characteristic (ROC) curves were plotted

for each of the NLP and ML techniques. The ROC curve illustrates an unbiased view of binary classifiers relative to one another, in terms of their performance at all of the classification thresholds. The closer the classifiers are to a right angle in the top right corner, the better their performance. In Figure 2, TF-IDF was used as the NLP technique. All of the classifiers are tightly grouped, however, SVM performed slightly better than the other four classifiers. In addition, KNN performed the worst out of all of the classifiers. In Figure 3, the Bag-of-Words NLP technique was used. Once again, all of the classifiers are tightly grouped, with SVM once again performing better than all of the other classifiers. The KNN and the DT classifiers performed the worst in prediction confidence. In Figure 4, the ROC for the Doc2Vec NLP technique is shown. With this preprocessor, the classifiers are much more spread and perform worse than the other two NLP techniques. The SVM classifier again performed the best, and DT performed the worst of all of the classifiers in the ROC.

When compared with existing antivirus solutions such as VirusTotal and Microsoft Windows Defender, the proposed approach demonstrates improved generalisation to unseen ransomware samples. Conventional antivirus engines are primarily based on signature-driven techniques, where detection is performed through hash lookups, byte-pattern matching, or rule-based heuristics. These techniques are highly effective for known threats, but they struggle against zero-day variants or polymorphic ransomware, in which slight modifications to the binary are sufficient to bypass signature recognition. In contrast, the proposed method leverages the statistical and semantic properties of printable strings extracted from executables, thereby enabling the detection of variants that share linguistic or structural similarities even if they have not been previously observed. This provides an inherent advantage over strictly signature-based solutions. However, one of the main challenges lies in the use of obfuscation techniques, such as string encryption, packing, code morphing, or polymorphism, which can significantly reduce the presence of discriminative string features. In such cases, the effectiveness of NLP-based feature extraction is diminished. Nevertheless, it is important to note that complete removal or encryption of all meaningful strings imposes trade-offs for attackers. Executables that fully obfuscate string data typically exhibit high entropy characteristics, which are easily detectable through entropy-based heuristics or dynamic analysis techniques. Consequently, while obfuscation may degrade performance in a purely string-based classification setting, it often exposes the malware to detection by complementary static or behavioural mechanisms.

VII. CONCLUSION

In this research report, the printable strings from executable files were analyzed using NLP and ML techniques on a dataset consisting of 2397 samples. The proposed method was successful in accurately classifying an executable as ransomware or benign in a short period of time. The best classifier is a

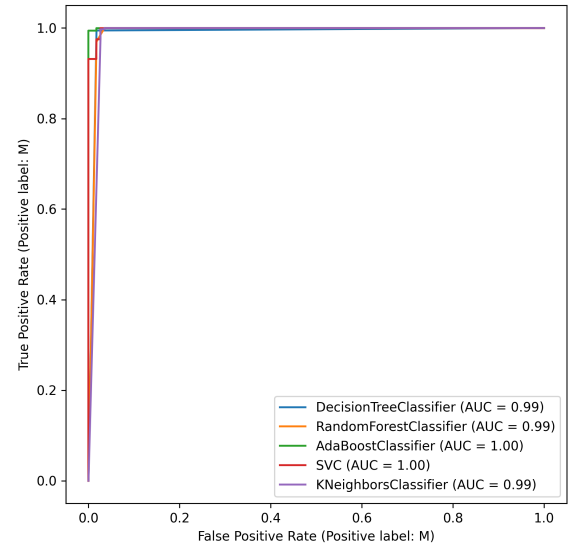


Fig. 4. Roc Curve for TF-IDF

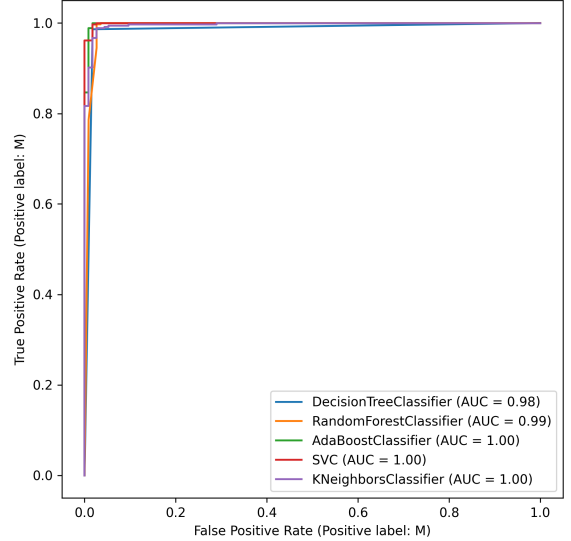


Fig. 5. Roc Curve for Bag-of-Words

decision tree with the TF-IDF pre-processing NLP technique, providing an accuracy of 99.38% and a prediction time of 0.0366ms per sample. Not only demonstrate that it is accurate, but also extremely fast. Dynamic analysis techniques, on the other hand, are too time-consuming to be practical in a real-world system; therefore, an accurate static analysis technique is much better suited to a system that aims to be deployed in the real world for real-time detection. A more in-depth analysis of the impact of the types of strings that are used was not explored; instead, all strings were considered in the same form in which they were extracted. Furthermore, the effects of executable compression on the accuracy of the system were not explored. Therefore, future work would evaluate the proposed technique on a dataset of greater size, which would

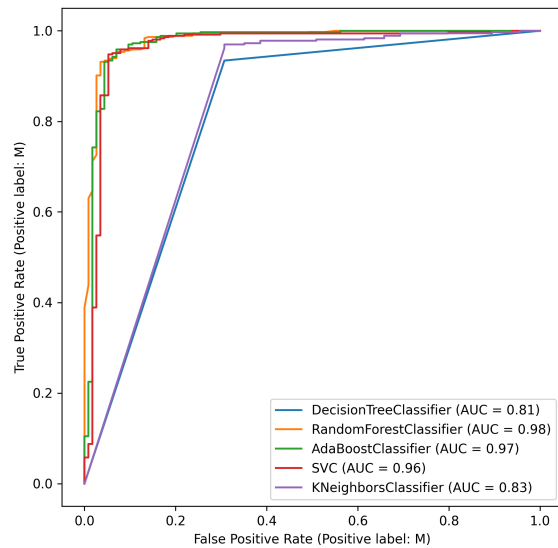


Fig. 6. Roc Curve for Doc2Vec

provide a more accurate analysis of the underlying merit. In addition, testing the technique on compressed executables and an analysis of the ideal types of strings to consider would also be beneficial. The proposed technique shows great promise as a method that can be used to quickly provide meaningful insights into whether an executable file contains ransomware.

The reinforcement-learning extension described in Section ?? (behavioural cloning from the best supervised model followed by REINFORCE fine-tuning) produced a test policy with accuracy 0.93 and macro F1 0.93. These results are reported to demonstrate feasibility of policy bootstrapping and continual adaptation; supervised TF-IDF classifiers still achieve the highest static performance in our experiments, but RL offers a path for online refinement under realistic deployment constraints.

REFERENCES

- [1] M. Sneha, A. Arya, and P. Agarwal, "Ransomware detection techniques in the dawn of artificial intelligence: A survey," *ACM International Conference Proceeding Series*, pp. 26–33, 2020.
- [2] Kaspersky, "Ransomware attacks and types – how encryption trojans differ." <https://www.kaspersky.com/resource-center/threats/ransomware-attacks-and-types>. Accessed: 2022-07-10.
- [3] I. SonicWall, "2025 sonicwallll cyber threat report," Tech. Rep. MSU-CSE-06-2, SonicWall, Inc., 1033 McCarthy Boulevard — Milpitas, CA 95035, January 2025.
- [4] IBM, "Cost of a data breach report 2024," tech. rep., IBM, July 2024.
- [5] B. Leddy, "How antivirus works?," <https://www.darktrace.com/en/blog/double-extortion-ransomware/>. Accessed: 2022-04-20.
- [6] C. Raghuraman, S. Suresh, S. Shivshankar, and R. Chapaneri, "Static and dynamic malware analysis using machine learning," *Advances in Intelligent Systems and Computing*, vol. 1045, pp. 793–806, 2020.
- [7] H. El Merabet and A. Hajraoui, "A survey of malware detection techniques based on machine learning," *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 1, pp. 366–373, 2019.
- [8] A. Singh, A. R. Ikuesan, and H. S. Venter, "Digital forensic readiness framework for ransomware investigation," in *Digital Forensics and Cyber Crime* (F. Breiting and I. Baggili, eds.), (Cham), pp. 91–105, Springer International Publishing, 2019.

- [9] A. Singh, R. Ikuesan, and H. Venter, "Ransomware detection using process memory," in *International Conference on Cyber Warfare and Security*, vol. 17, pp. 413–422, 2022.
- [10] J. Hwang, J. Kim, S. Lee, and K. Kim, "Two-Stage Ransomware Detection Using Dynamic Analysis and Machine Learning Techniques," *Wireless Personal Communications*, vol. 112, no. 4, pp. 2597–2609, 2020.
- [11] O. Ahmed and O. Al-Dabbagh, "Ransomware Detection System Based on Machine Learning," *Journal of Education and Science*, vol. 30, no. 5, pp. 86–102, 2021.
- [12] E. M. B. Karbab and M. Debbabi, "MalDy: Portable, data-driven malware detection using natural language processing and machine learning techniques on behavioral analysis reports," *Digital Investigation*, vol. 28, pp. S77–S87, 2019.
- [13] M. Mimura and R. Ito, "Applying NLP techniques to malware detection in a practical environment," *International Journal of Information Security*, vol. 21, no. 2, pp. 279–291, 2022.
- [14] T. Lowe, C. Fisher, and J. Collins, "Advanced Ransomware Detection and Classification via Semantic Analysis of Memory Opcode Patterns," pp. 1–18.
- [15] B. M. Khammas, "Ransomware Detection using Random Forest Technique," *ICT Express*, vol. 6, no. 4, pp. 325–331, 2020.
- [16] A. Singh, R. A. Ikuesan, and H. Venter, "MalFe—Malware Feature Engineering Generation Platform," *Computers*, vol. 12, no. 10, pp. 1–20, 2023.
- [17] Turing, "A guide on word embeddings in nlp." <https://www.turing.com/kb/guide-on-word-embeddings-in-nlp>. Accessed: 2022-12-07.
- [18] Cuckoo, "Cuckoo sandbox strings." <https://github.com/cuckoosandbox/cuckoo/blob/master/cuckoo/processing/strings.py>, 2022.
- [19] Capital One, "Understanding tf-idf for machine learning." <https://www.capitalone.com/tech/machine-learning/understanding-tf-idf/>. Accessed: 2022-08-14.
- [20] J. Brownlee, "A gentle introduction to the bag-of-words model," Aug 2019.
- [21] Jason Brownlee, "A gentle introduction to the bag-of-words model." <https://machinelearningmastery.com/gentle-introduction-bag-words-model/>. Accessed: 2022-12-12.
- [22] G. Shperber, "A gentle introduction to doc2vec." <https://medium.com/wisio/a-gentle-introduction-to-doc2vec-db3e8c0cce5e>. Accessed: 2022-08-17.
- [23] Shuzhan Fan, "Understanding word2vec and doc2vec." <https://shuzhanfan.github.io/2018/08/understanding-word2vec-and-doc2vec/>. Accessed: 2022-12-13.
- [24] D. Nortier, "Rds - ransomware detection using strings." <https://github.com/ICFL-UP/RDS>, 2022.